

Weber's equation for the fluctuation function:

$$f'' + \left(\frac{1}{2} - 2\lambda - \frac{z^2}{4}\right)f = 0$$

Claude, with Daniel Murfet

2026-09-06

Abstract

Unit 8 of the grammar-paper §4 autoformalisation. The substitution $S_\lambda(a) = e^{\beta a^2/8} f(a\sqrt{\beta/2})$ turns the fluctuation ODE (iii) into *Weber's equation* $f''(z) + (\frac{1}{2} - 2\lambda - \frac{z^2}{4})f(z) = 0$ (`prop:fluctuation_weber`), the standard parabolic cylinder equation with order $\nu = -2\lambda$ — so the Weber order depends only on the RLCT λ , not on the inverse temperature β . Proved from the `HasDerivAt` derivative ladder alone, no special-function library. Zero `sorry/axiom`.

1 Setting

The closed form (unit 7) identified S_λ with a parabolic cylinder function $D_{-2\lambda}$. This proposition is the ODE face of that identification, and unlike the closed form it needs *no* D_ν machinery: it is a pure second-derivative computation on top of the already-formalised derivative ladder ($S'_\lambda = \beta S_{\lambda+1/2}$, $S''_\lambda = \beta^2 S_{\lambda+1}$) and the ODE (iii) $S''_\lambda = \frac{a\beta}{2} S'_\lambda + \lambda\beta S_\lambda$. It is the last non-operator item of §4 that stands independently of D_ν .

2 The thing formalised

```
noncomputable def weberSub ( lam z : ) : :=
  Real.exp (-z ^ 2 / 4) * fluctuation lam (z * Real.sqrt (2 / ))

theorem fluctuation_weber ( lam z : ) (h : 0 < ) (hlam : 0 < lam) :
  deriv (deriv (fun z => weberSub lam z)) z
  + (1 / 2 - 2 * lam - z ^ 2 / 4) * weberSub lam z = 0
```

We formalise the substitution solved for f : $f(z) = e^{-z^2/4} S_\lambda(z\sqrt{2/\beta})$ (with $\sqrt{2/\beta} = 1/\sqrt{\beta/2}$, so $z = a\sqrt{\beta/2}$).

3 Proof strategy

Write $k = \sqrt{2/\beta}$, so the argument is $a = zk$ and $\beta k^2 = 2$. The plan (GPT-6 Astra) is “prove the first derivative as an explicit function everywhere, then differentiate that function”:

1. Build `HasDerivAt` for the two fluctuation compositions $a \mapsto S_\lambda(ak)$ and $a \mapsto \beta S_{\lambda+1/2}(ak)$ by `HasDerivAt.comp` with the linear map $x \mapsto xk$ (giving the extra factor k), and for the Gaussian $e^{-z^2/4}$ (derivative $-\frac{z}{2}e^{-z^2/4}$).

2. Product rule gives $f_1(w) = \text{deriv } f(w) = e^{-w^2/4}(-\frac{w}{2}S_\lambda(wk) + k\beta S_{\lambda+1/2}(wk))$ as a function; `funext` of `HasDerivAt.deriv` turns this into the *function equality* `deriv f = f1`.
3. Differentiate f_1 once more (product+sum+const-mul rules) to get `deriv(deriv f)(z) = f2,z`.
4. Substitute the ODE (iii) for the $\beta^2 S_{\lambda+1}(zk)$ term. The whole expression factors as $e^{-z^2/4}(\beta k^2 - 2)(\lambda S_\lambda(zk) + \frac{zk}{2}\beta S_{\lambda+1/2}(zk))$, which vanishes because $\beta k^2 = 2$.

The three fluctuation values $S_\lambda(zk)$, $S_{\lambda+1/2}(zk)$, $S_{\lambda+1}(zk)$ stay opaque atoms throughout; the ODE eliminates the third and `ring` exposes the common factor $\beta k^2 - 2$.

4 Roadblocks and resolutions

Three `HasDerivAt` mechanics, each a reusable lesson:

- **comp higher-order unification splits at the wrong place.** Composing the derivative of $a \mapsto \beta S_{\lambda+1/2}(a)$ with the linear map, `exact (...).comp` with the goal function $\lambda x. \beta S_{\lambda+1/2}(xk)$ let the elaborator peel the outer function as $(\beta \cdot -)$ rather than the intended $S_{\lambda+1/2}$ -composition (error: expected `HasDerivAt (HMul.hMul β) ...`). Fix: state that goal with the function written as an explicit `Function.comp ((λa. β Sλ+1/2(a)) ∘ (λx. xk))`, so no HOU is needed.
- **Combinator derivative values need normalising.** The product/sum/const_mul combinators produce a derivative in a fixed (un-factored) shape. `convert ...using 1` stops at the `ContinuousLinearMap.smulRight` wrapper (leaving a goal `ring` cannot touch), and `using 2` leaves a spurious `Module`-instance diamond goal (`Semiring.toModule` vs `(NormedAlgebra.toNormedSpace ℝ).toModule`). The clean tool is `HasDerivAt.congr_deriv h` (by `ring`), which changes only the derivative value and lets `ring` prove the raw=clean equality, while `exact` reconciles the instance diamond by `defeq`.
- `simp` `...using (comp result)` fails on the same instance diamond (its final match is stricter than `exact`); using `exact` on the bound composition succeeds.

5 What was Mathlib, what was new

Everything analytic is Mathlib: `HasDerivAt.comp/.mul/.add/.const_mul/.exp, hasDerivAt_pow, HasDerivAt.congr_deriv, HasDerivAt.deriv, and Real.sq_sqrt`. The new content is entirely the assembly: the explicit first- and second-derivative functions, and the observation that the ODE substitution collapses the second-order combination to a $(\beta k^2 - 2)$ multiple, which the single square-root identity $\beta k^2 = 2$ kills.

6 Lessons

For a change-of-variables ODE identity, differentiate the substituted function as an *explicit function* (via `funext ∘ HasDerivAt.deriv`) rather than manipulating `deriv` pointwise — the second derivative is then just “differentiate a named function again”. And `HasDerivAt.congr_deriv` is the right way to reconcile a combinator’s raw derivative value with a hand-chosen clean form: it sidesteps both the `smulRight` wrapper and the `Module`-instance diamond that defeat `convert` and `simp`.

7 Follow-ups

The remaining §4 items rest on the positive-order D_n (Hermite×Gaussian), which Mathlib does not have as a special function though it *does* have the (probabilist) Hermite polynomials: `eq:Dn_hermite` ($D_n(z) = e^{-z^2/4}\text{He}_n(z)$ after the physicist/probabilist bridge) and `lem:qho` (the quantum harmonic oscillator). The operator identity $[b, b^\dagger] = 1$ needs a Weyl algebra framework.